FONDAMENTI DI INFORMATICA
A.A. 2026/27 - Politecnico di Milano

Sez. Prof. Salvatore Andolina
Resp. Laboratorio: Prof. Riccardo Medana

Laboratorio di Programmazione in C

Laboratorio 1 - 25 settembre 2026

Es. 1.1 // Shift circolare di tre variabili

■ *printf() e scanf()*

Provare a compilare ed eseguire il programma contenuto nel file `shift_wrong.c` fornito con il testo dell'esercizio:

shift_wrong.c

```c
#include <stdio.h>

int Main() {
    int a, b c;

    printf("Inserire il valore di a: ");
    scanf("%d", a);

    printf("\nInserire il valore di b: ");
    scanf("%d", b)

    printf("\nInserire il valore di c: ")
    scanf("d", c);

    /* Adesso faccio lo shift *
     b = a
     c = b
     a = c

     printf("\nIl valore di a è: d", a);
     printf("\nIl valore di b è: d", b);
     printf("\nIl valore di c è: d\n", d);

     return 0;
}
```

Il codice del programma contiene svariati errori di compilazione e di esecuzione: correggerlo in modo che venga compilato con successo e che esegua lo shift circolare del contenuto delle tre variabili, cioè metta il contenuto di **a** in **b**, il contenuto di **b** in **c**, il contenuto di **c** in **a**.

Es. 1.2 // Ordinamento e differenze

■ *printf() e scanf(), condizioni*

Provare a compilare ed eseguire il programma contenuto nel file `ordinamento_differenze_wrong.c` fornito con il testo dell'esercizio:

ordinamento_differenze_wrong.c

```c
#include <stdio.h>

```

```c
int main() (
    Int x, y, z, min, mid max;

    printf("Inserisci tre numeri interi: ");
    scanf("%d %d %d", x, y, z)

    if (x > y)
        if (y > z)
            max = x; mid = y; min = z;
        else if (x > z)
            max = y; mid = z; min = x;
        else
            max = z; mid = x; min = y;
    Else
        if (x > z)
            max = y; mid = x; min = z;
        else if (y > z)
            max = y; mid = z; min = x;
        else
            max = x; mid = y; min = z;

    printf("Numeri in ordine crescente: %d d %d\n", min, mid, max);

    diff = max – min;
    printf("Differenza tra massimo e minimo: %d\n", diff);

    return 0
}
```

Il codice del programma contiene svariati errori di compilazione e di esecuzione: il programma dovrebbe leggere tre numeri interi, stamparli in ordine crescente e poi stampare la differenza tra il maggiore e il minore. Correggerlo in modo che venga compilato ed eseguito con successo.

Es. 1.3 // **Somma di frazioni**

■ *printf() e scanf()*

Scrivere un programma che chieda all'utente di inserire due frazioni di segno positivo $a = \frac{a_1}{b_1}$ e $b = \frac{a_2}{b_2}$, calcoli il risultato della somma di frazioni $c = a + b$ e lo stampi a video (c dovrà essere in formato frazione).

Le frazioni vanno inserite dall'utente nel formato **numeratore / denominatore**. Per una variante più semplice è possibile anche inserire i valori di numeratore e denominatore chiedendoli singolarmente. Non è necessario ridurre ai minimi termini il risultato della somma.

Es. 1.4 // **Bisestile o no?**

■ *printf() e scanf(), condizioni*

Scrivere un programma che verifichi se un anno inserito dall'utente è bisestile. Un anno è considerato bisestile se soddisfa almeno una delle seguenti condizioni:

- L'anno è divisibile per 4 e non è divisibile per 100
- L'anno è divisibile per 400.

Il programma deve gestire correttamente l'input dell'utente, assicurandosi che l'anno inserito sia un numero intero positivo. In caso di input non valido, il programma deve fornire un messaggio di errore appropriato.

Es. 1.5 // **Indagine su due numeri**

■ *printf() e scanf(), condizioni*

Scrivere un programma che richiede all'utente di inserire due numeri interi. Successivamente, il programma deve eseguire le seguenti operazioni:

- verificare se il risultato della loro moltiplicazione è un numero pari o dispari senza effettuare il prodotto;
- verificare se il primo numero è multiplo del secondo;
- determinare quale dei due numeri è maggiore, oppure se sono uguali;
- ipotizzando che i due numeri rappresentino le coordinate x e y di un punto in un piano cartesiano, determinare in quale quadrante si trova il punto, oppure se si trova sull'asse x o y o nell'origine.

Il programma deve gestire correttamente l'input dell'utente, assicurandosi che i valori inseriti siano numeri interi. In caso di input non valido, il programma deve fornire un messaggio di errore appropriato.

Es. 1.6 // **"Come disporsi in aula"**

■ *Cicli, condizioni, printf() e scanf()*

L'aula del Laboratorio è composta da `x` file di `y` postazioni per un totale di `p` posti (dove `p = x * y`).

Ogni aula ha al massimo 15 file, ciascuna composta da un massimo di 20 postazioni.

Al corso partecipano `n` studenti. Questi devono disporsi in modo che:

- ci stiano tutti;
- ogni due file piene ci sia una fila completamente vuota;
- si inizi a riempire l'aula a partire dalla prima fila davanti alla cattedra;
- ogni fila di posti occupabile venga riempita senza lasciare spazi vuoti tra uno studente e l'altro.

Scrivere un programma che, una volta ricevuti i valori `x`, `y` e `n` dall'utente:

- verifichi che `x`, `y` e `n` siano tutti interi positivi diversi da zero, e che rispettino le caratteristiche richieste. Se i valori non sono corretti il programma termina comunicando l'errore all'utente;
- controlli a monte che tutti gli studenti possano accomodarsi in aula secondo le regole indicate. Se non c'è spazio per tutti gli studenti il programma termina comunicando l'errore all'utente;
- stampi a video la pianta dell'aula a partire dalla prima fila, indicando con il carattere `X` i posti occupati e con il carattere `0` i posti liberi.

Es. 1.7 // **Lettura del pensiero**

■ *printf() e scanf(), condizioni, cicli*

Scrivere un programma che simula un semplice numero di mentalismo. Il programma chiede all'utente di pensare al numero del suo giorno di nascita, senza rivelarlo. Per cinque volte, il programma mostra all'utente una serie di numeri (apparentemente casuali) tra 1 e 31, chiedendogli se il suo numero è presente fra questi. A ogni iterazione, l'utente risponde inserendo `0` ("no") o `1` ("sì") e premendo `Invio`.

Al termine il programma stampa a video il numero pensato dall'utente.

Utilizzare i cicli per la generazione delle serie di numeri da mostrare all'utente e la ripetizione di parti comuni. Il programma deve gestire correttamente l'input dell'utente, assicurandosi che i valori inseriti siano sempre `0` o `1`. In caso di input non valido, il programma si interrompe e fornisce un messaggio di errore appropriato. Per l'implementazione del programma consultare la spiegazione del trucco riportata di seguito.

Spiegazione del trucco

Le serie di numeri mostrate all'utente **non sono casuali**, ma sono costruite in modo da coprire tutti i numeri tra 1 e 31. Le serie da generare sono le seguenti:

- `1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31`
- `2, 3, 6, 7, 10, 11, 14, 15, 18, 19, 22, 23, 26, 27, 30, 31`
- `4, 5, 6, 7, 12, 13, 14, 15, 20, 21, 22, 23, 28, 29, 30, 31`
- `8, 9, 10, 11, 12, 13, 14, 15, 24, 25, 26, 27, 28, 29, 30, 31`
- `16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31`

Il numero pensato dall'utente si ricava prendendo tutte le serie che lo contengono e sommando **il primo numero** presente in ciascuna di queste.

Il trucco si basa sulla rappresentazione binaria dei numeri: ogni numero tra 1 e 31 può essere rappresentato con 5 bit (cifre binarie). Ciascuna delle cinque serie di numeri mostrate corrisponde a un bit specifico (unità, due, quattro, otto, sedici - lo si può notare dal primo valore di ogni serie). Ogni serie contiene quindi tutti i numeri tra 1 e 31 che hanno il bit corrispondente impostato a **1**. Ad esempio, la prima serie contiene tutti i numeri dispari (1, 3, 5, ..., 31), la seconda serie contiene tutti i numeri che, in binario, hanno il secondo bit impostato a **1** (2, 3, 6, 7, ...), e così via. Se sommiamo i valori corrispondenti ai bit per cui l'utente ha risposto "sì" (**1**) possiamo ottenere il numero pensato dall'utente.

Per esempio: il numero 19 è presente nella prima, seconda e quinta serie, quindi basta calcolare **`00001`** + **`00010`** + **`10000`** = **`10011`**, che in sistema decimale corrisponde a $2^4 + 0 + 0 + 2^1 + 2^0 = 16 + 2 + 1 = 19$.

Esempio di esecuzione

```
Pensa al numero del tuo giorno di nascita (non dirmelo!).

Ora ti mostrerò alcune serie di numeri, dovrai dirmi se il tuo numero è presente in ciascuna di esse.

1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31
Il tuo numero è in questa serie? (0=no, 1=si): 1

2, 3, 6, 7, 10, 11, 14, 15, 18, 19, 22, 23, 26, 27, 30, 31
Il tuo numero è in questa serie? (0=no, 1=si): 1

4, 5, 6, 7, 12, 13, 14, 15, 20, 21, 22, 23, 28, 29, 30, 31
Il tuo numero è in questa serie? (0=no, 1=si): 0

8, 9, 10, 11, 12, 13, 14, 15, 24, 25, 26, 27, 28, 29, 30, 31
Il tuo numero è in questa serie? (0=no, 1=si): 0

16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31
Il tuo numero è in questa serie? (0=no, 1=si): 1

Il numero a cui stavi pensando è 19.
```